

Likhitha_AAI_500_M2_Assignment

May 18, 2026

1 Assignment 2.1

Name: Likhitha Potluri

Date: 15-05-2026

For this assignment, you will refer to the textbook to solve the practice exercises. **Use Python to answer any coding problems (not R, even if indicated in your textbook).** Use Jupyter Notebook, Google Colab, or a similar software program to complete your assignment. Submit your answers as a **PDF or HTML** file. As a best practice, always label your axes and provide titles for any graphs generated on this assignment. Round all quantitative answers to 2 decimal places.

1.1 Problem # 2.1.

For the rain simulation example in Section 2.1.1, but with probability of rain 0.30 on any given day, simulate the outcome (a) on the next day, (b) the next 10 days. (c) Simulate the proportion of days of rain for the next (i) 100 days, (ii) 10,000 days, (iii) 1,000,000 days. Use the simulation to explain the long-run relative frequency definition of probability.

(a) Your answer goes here

```
[2]: import numpy as np

np.random.seed(42)
p_rain = 0.30

next_day = np.random.binomial(n=1, p=p_rain)
print("Next day outcome:", "Rain" if next_day == 1 else "No Rain")
```

Next day outcome: No Rain

(b) Your answer goes here

```
[3]: import numpy as np

np.random.seed(42)
p_rain = 0.30

next_10_days = np.random.binomial(n=1, p=p_rain, size=10)
```

```
print("Next 10 days outcomes:")
for i, x in enumerate(next_10_days, start=1):
    print(f"Day {i}: {'Rain' if x == 1 else 'No Rain'}")
```

Next 10 days outcomes:

Day 1: No Rain
 Day 2: Rain
 Day 3: Rain
 Day 4: No Rain
 Day 5: No Rain
 Day 6: No Rain
 Day 7: No Rain
 Day 8: Rain
 Day 9: No Rain
 Day 10: Rain

(c) Your answer goes here

```
[4]: import numpy as np

np.random.seed(42)
p_rain = 0.30

for n in [100, 10_000, 1_000_000]:
    sim = np.random.binomial(n=1, p=p_rain, size=n)
    prop_rain = sim.mean()
    print(f"Proportion of rainy days over {n:,} days: {prop_rain:.5f}")

print("\nExplanation:")
print("As the number of days increases, the simulated proportion gets closer to 0.30.")
print("This demonstrates long-run relative frequency: probability is the long-run")
print("proportion of times an event occurs over many repeated trials.")
```

Proportion of rainy days over 100 days: 0.30000
 Proportion of rainy days over 10,000 days: 0.28890
 Proportion of rainy days over 1,000,000 days: 0.30029

Explanation:

As the number of days increases, the simulated proportion gets closer to 0.30.
 This demonstrates long-run relative frequency: probability is the long-run
 proportion of times an event occurs over many repeated trials.

1.2 Problem # 2.2.

Data analysts often implement statistical inference methods by setting the probability of a correct inference equal to 0.95. Let A denote the event that an inference for the population about men

is correct. Let B represent the event of a corresponding inference about women being correct. Suppose that these are independent events.

- (a) Find the probability that (i) *both* inferences are correct, (ii) *neither* inference is correct.
- (b) Construct the probability distribution for Y = number of correct inferences.
- (c) With what probability would each inference need to be correct in order for the probability to be 0.95 that *both* are correct?

(a) Your answer goes here

```
[5]: pA = 0.95
      pB = 0.95

      p_both_correct = pA * pB
      p_neither_correct = (1 - pA) * (1 - pB)

      print("(a)(i) P(both correct) =", round(p_both_correct, 4))
      print("(a)(ii) P(neither correct) =", round(p_neither_correct, 4))
```

(a)(i) $P(\text{both correct}) = 0.9025$

(a)(ii) $P(\text{neither correct}) = 0.0025$

(b) Your answer goes here

```
[6]: pA = 0.95
      pB = 0.95

      # Independent Bernoulli trials
      pY0 = (1 - pA) * (1 - pB)           # neither correct
      pY2 = pA * pB                       # both correct
      pY1 = pA * (1 - pB) + (1 - pA) * pB # exactly one correct

      dist = {
          0: round(pY0, 4),
          1: round(pY1, 4),
          2: round(pY2, 4)
      }

      print("(b) Distribution of Y (number of correct inferences):")
      for y, p in dist.items():
          print(f"P(Y={y}) = {p}")

      print("Check sum =", round(pY0 + pY1 + pY2, 4))
```

(b) Distribution of Y (number of correct inferences):

$P(Y=0) = 0.0025$

$P(Y=1) = 0.095$

$P(Y=2) = 0.9025$

Check sum = 1.0

(c) Your answer goes here

```
[7]: import math

target = 0.95
p_required = math.sqrt(target)

print("(c) Required probability for each inference =", round(p_required, 6))
print("As percent =", round(p_required * 100, 3), "%")
```

(c) Required probability for each inference = 0.974679

As percent = 97.468 %

1.3 Problem # 2.4.

A wine connoisseur is asked to match five glasses of red wine with the bottles from which they came, representing five different grape types.

(a) Set up a sample space for the five guesses.

(b) With random guessing, find the probability of getting all five correct.

(a) Your answer goes here

The sample space consists of all possible permutations of the five glasses, `sample_space = list(itertools.permutations([1, 2, 3, 4, 5]))`, the length of sample space is 120. Example: (3, 2, 4, 1, 5), (3, 2, 4, 5, 1), (3, 2, 5, 1, 4), (3, 2, 5, 4, 1)

(b) Your answer goes here

```
[10]: import itertools
sample_space = list(itertools.permutations([1, 2, 3, 4, 5]))
print("Sample space size:", len(sample_space))
# (b) With random guessing, find the probability of getting all five correct.
# The probability of getting all five correct with random guessing is 1 / □
    ↳ number of permutations.
probability_all_correct = 1 / len(sample_space)
print("Probability of getting all five correct with random guessing:", □
    ↳ round(probability_all_correct, 2))
```

Sample space size: 120

Probability of getting all five correct with random guessing: 0.01

1.4 Problem # 2.15.

Each week an insurance company records Y = number of payments because of a home burning down. State conditions under which we would expect Y to approximately have a Poisson distribution.

Your answer goes here

We would expect Y to approximately have a Poisson distribution under the following conditions:

1. The events (home burnings) occur independently of each other.
2. The probability of a home burning down in a short time interval is proportional to the length of the interval.
3. The probability of more than one home burning down in a very short time interval is negligible.
4. The rate of home burnings is constant over time.

In simpler terms, the events should be rare, random, and independent, and the average rate of events should be relatively constant.

1.5 Problem # 2.16.

Each day a hospital records the number of people who come to the emergency room for treatment.

- (a) In the first week, the observations from Sunday to Saturday are 10, 8, 14, 7, 21, 44, 60. Do you think that the Poisson distribution might describe the random variability of this phenomenon adequately. Why or why not?
- (b) Would you expect the Poisson distribution to better describe, or more poorly describe, the number of weekly admissions to the hospital for a rare disease? Why?

(a) Your answer goes here

Probably not very well for this week's daily data. A Poisson model assumes a roughly constant event rate over equal time intervals and has variance approximately equal to the mean. Here the counts increase sharply late in the week (especially 44 and 60), suggesting the rate is not constant across days (possible weekday/weekend effects or other time-pattern effects). The spread also looks much larger than what a simple Poisson with one fixed rate would imply (overdispersion). So a single Poisson model for all days is likely inadequate.

(b) Your answer goes here

It would likely describe them better. For rare diseases, events are typically infrequent and can often be approximated as independent occurrences over time, which matches Poisson assumptions more closely. Aggregating to weekly totals can also smooth day-to-day patterns, making a Poisson model more reasonable (as long as the underlying weekly rate is fairly stable).

1.6 Problem # 2.17.

An instructor gives a course grade of B to students who have total score on exams and homeworks between 800 and 900, where the maximum possible is 1000. If the total scores have approximately a normal distribution with mean 830 and standard deviation 50, about what proportion of the students receive a B?

Your answer goes here

```
[11]: from scipy.stats import norm

mu = 830
sigma = 50
lower = 800
upper = 900
```

```

proportion_B = norm.cdf(upper, loc=mu, scale=sigma) - norm.cdf(lower, loc=mu,
↪scale=sigma)

print(f"Proportion receiving B: {proportion_B:.4f}")
print(f"Percentage receiving B: {proportion_B*100:.2f}%")

```

Proportion receiving B: 0.6450
Percentage receiving B: 64.50%

1.7 Problem # 2.20.

Create a data file with the income values in the `Income` data file linked on the Module 2 Readings page.

- Construct a histogram or a smooth-curve approximation for the *pdf* of income in the corresponding population by plotting results using the density function in R (explained in Exercise 1.18).
- Of the probability distributions studied in this chapter, which do you think might be most appropriate for these data? Why? Plot the probability function of that distribution having the same mean and standard deviation as the income values. Does it seem to describe the income distribution well?

(a) Your answer goes here

```

[13]: # Step 0: Create a clean income-only data file from Income.dat
      # Output file: income_values.csv

import requests
import pandas as pd

url = "http://stat4ds.rwth-aachen.de/data/Income.dat"
tokens = requests.get(url, timeout=30).text.strip().split()

def is_float(x):
    try:
        float(x)
        return True
    except:
        return False

# If first token is non-numeric, assume header like: income education race
start = 0
if not is_float(tokens[0]):
    start = 3

# Parse triplets: income, education, race
income_vals = []
for i in range(start, len(tokens), 3):

```

```

        if i < len(tokens) and is_float(tokens[i]):
            income_vals.append(float(tokens[i]))

income_df = pd.DataFrame({"income": income_vals})
income_df.to_csv("income_values.csv", index=False)

print("Saved:", "income_values.csv")
print("Rows:", len(income_df))
print(income_df.head())

```

Saved: income_values.csv

Rows: 80

	income
0	16.0
1	18.0
2	26.0
3	16.0
4	34.0

[14]: # (a) Histogram + smooth-curve approximation (KDE) for income density

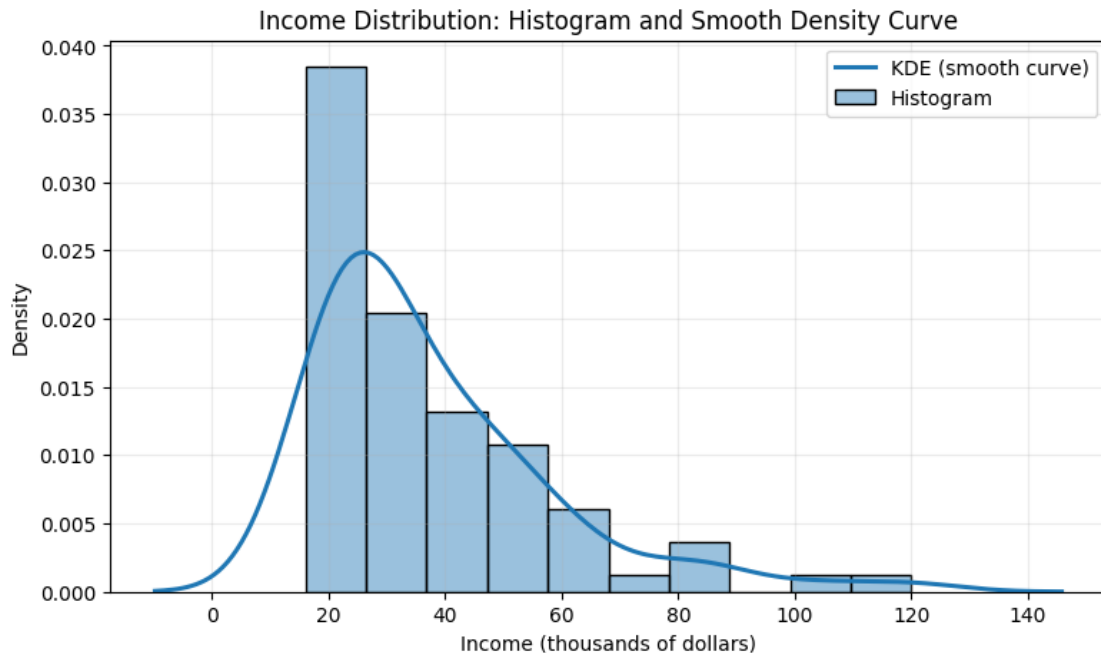
```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("income_values.csv")

plt.figure(figsize=(9, 5))
sns.histplot(df["income"], bins="fd", stat="density", edgecolor="black",
             alpha=0.45, label="Histogram")
sns.kdeplot(df["income"], bw_adjust=1.0, linewidth=2.2, label="KDE (smooth
             curve)")
plt.title("Income Distribution: Histogram and Smooth Density Curve")
plt.xlabel("Income (thousands of dollars)")
plt.ylabel("Density")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

```



(b) Your answer goes here

```
[15]: # (b) Fit a candidate distribution using same mean and SD as income values
# Recommended candidate for income: Lognormal (positive + right-skewed)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import lognorm, norm

df = pd.read_csv("income_values.csv")
x_data = df["income"].dropna().values

m = x_data.mean()
s = x_data.std(ddof=1)

# Lognormal parameters from mean m and sd s
sigma2 = np.log(1 + (s**2 / m**2))
sigma = np.sqrt(sigma2)
mu = np.log(m) - 0.5 * sigma2

x = np.linspace(max(0.001, x_data.min()*0.5), x_data.max()*1.3, 600)

lognorm_pdf = lognorm.pdf(x, s=sigma, scale=np.exp(mu))
normal_pdf = norm.pdf(x, loc=m, scale=s) # optional comparison
```

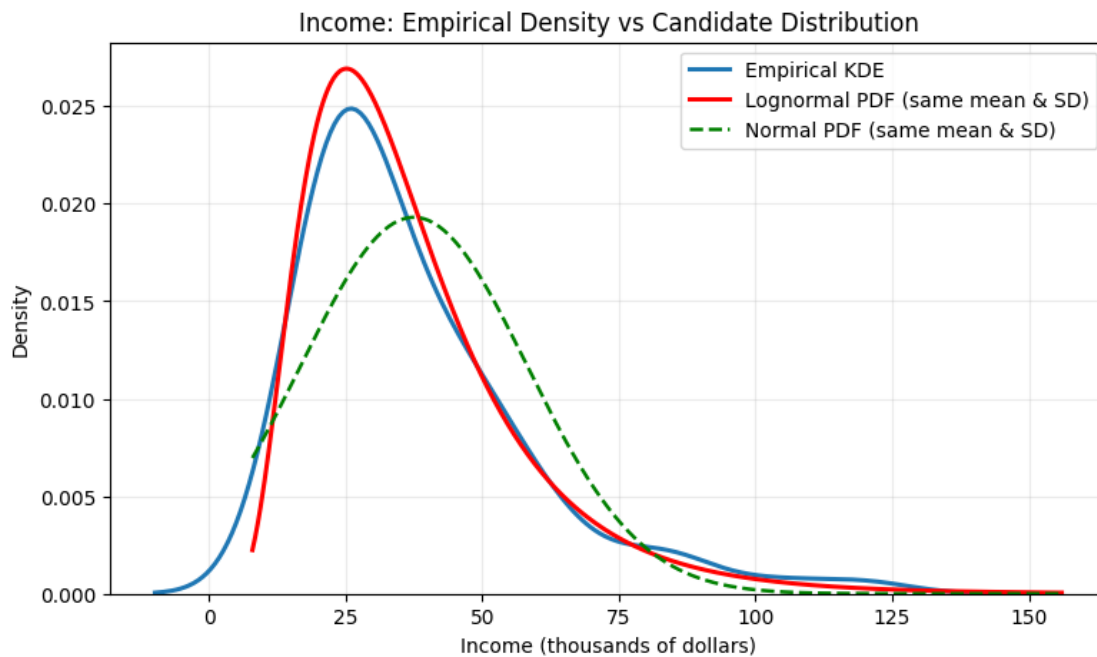


```

plt.figure(figsize=(9, 5))
sns.kdeplot(x_data, bw_adjust=1.0, linewidth=2.2, label="Empirical KDE")
plt.plot(x, lognorm_pdf, "r-", linewidth=2.2, label="Lognormal PDF (same mean & SD)")
plt.plot(x, normal_pdf, "g--", linewidth=1.8, label="Normal PDF (same mean & SD)")
plt.title("Income: Empirical Density vs Candidate Distribution")
plt.xlabel("Income (thousands of dollars)")
plt.ylabel("Density")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

print(f"Sample mean = {m:.3f}")
print(f"Sample SD   = {s:.3f}")
print("Suggested distribution: Lognormal (income is positive and typically right-skewed).")

```



Sample mean = 37.525

Sample SD = 20.673

Suggested distribution: Lognormal (income is positive and typically right-skewed).

1.8 Problem # 2.21.

Plot the gamma distribution by fixing the shape parameter $k = 3$ and setting the scale parameter $= 0.5, 1, 2, 3, 4, 5$. What is the effect of increasing the scale parameter? (See also Exercise 2.48.)

Your answer goes here

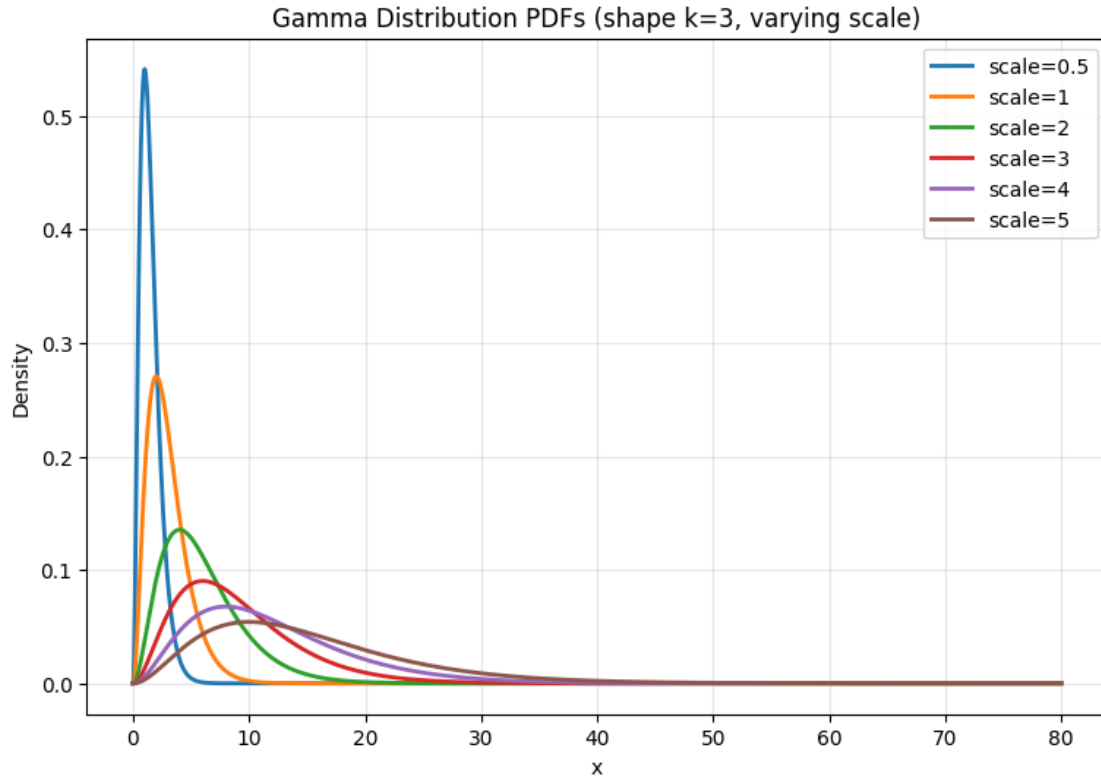
```
[16]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gamma

k = 3
scales = [0.5, 1, 2, 3, 4, 5]

x = np.linspace(0, 80, 2000)

plt.figure(figsize=(9, 6))
for theta in scales:
    y = gamma.pdf(x, a=k, scale=theta)
    plt.plot(x, y, linewidth=2, label=f"scale={theta}")

plt.title("Gamma Distribution PDFs (shape k=3, varying scale)")
plt.xlabel("x")
plt.ylabel("Density")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



1.9 Problem # 2.22.

Consider the mammogram diagnostic example in Section 2.1.4.

- Show that the joint probability distribution of diagnosis and disease status is as shown in Table 2.6. Given that a diagnostic test result is positive, explain how this joint distribution shows that the 12% of incorrect diagnoses for the 99% of women not having breast cancer swamp the 86% of correct diagnoses for the 1% of women actually having breast cancer.
- The first test for detecting HIV-positive status had a sensitivity of 0.999 and specificity of 0.9999. Explain what these mean. If at that time 1 in 10,000 men were truly HIV-positive, find the positive predictive value. Based on this example, explain the potential disadvantage of routine diagnostic screening of a population for a rare disease.

Table 2.6. Joint probability distribution for disease status and diagnosis of breast-cancer mammogram, based on conditional probabilities in Table 2.1.

Disease Status	Positive (+)	Negative (−)	Total
Yes (D)	0.0086	0.0014	0.01
No (D^c)	0.1188	0.8712	0.99

Your answer goes here

[17]: *# Part (a): Mammogram joint distribution + interpretation with Bayes quantities*

Given (from section/table assumptions):

$P(D) = 0.01$, $P(Dc)=0.99$

$P(+|D)=0.86$, $P(-|D)=0.14$

$P(+|Dc)=0.12$, $P(-|Dc)=0.88$

$P_D = 0.01$

$P_Dc = 0.99$

$P_pos_given_D = 0.86$

$P_neg_given_D = 0.14$

$P_pos_given_Dc = 0.12$

$P_neg_given_Dc = 0.88$

Joint probabilities

$P_D_and_pos = P_D * P_pos_given_D$

$P_D_and_neg = P_D * P_neg_given_D$

$P_Dc_and_pos = P_Dc * P_pos_given_Dc$

$P_Dc_and_neg = P_Dc * P_neg_given_Dc$

print("Joint table entries:")

print(f"P(D and +) = {P_D_and_pos:.4f}") # 0.0086

print(f"P(D and -) = {P_D_and_neg:.4f}") # 0.0014

print(f"P(Dc and +) = {P_Dc_and_pos:.4f}") # 0.1188

print(f"P(Dc and -) = {P_Dc_and_neg:.4f}") # 0.8712

Positive-test totals and predictive values

$P_pos = P_D_and_pos + P_Dc_and_pos$

$PPV = P_D_and_pos / P_pos$ # $P(D|+)$

$false_positive_share = P_Dc_and_pos / P_pos$ # $P(Dc|+)$

print(f"\nP(+) = {P_pos:.4f}")

print(f"P(D|+) (PPV) = {PPV:.4%}")

print(f"P(Dc|+) = {false_positive_share:.4%}")

print(f"False positives are {P_Dc_and_pos / P_D_and_pos:.2f} times true positives.")

Joint table entries:

P(D and +) = 0.0086

P(D and -) = 0.0014

P(Dc and +) = 0.1188

P(Dc and -) = 0.8712

P(+) = 0.1274

P(D|+) (PPV) = 6.7504%

P(Dc|+) = 93.2496%

False positives are 13.81 times true positives.

```
[18]: # Part (b): HIV test example
# Sensitivity = 0.999, specificity = 0.9999, prevalence = 1/10000

sensitivity = 0.999
specificity = 0.9999
prevalence = 1/10000 # 0.0001

false_positive_rate = 1 - specificity

# Positive predictive value:
# PPV = P(HIV+ | test+) = [sensitivity*prevalence] /
# [sensitivity*prevalence +
# ↪ false_positive_rate*(1-prevalence)]
ppv = (sensitivity * prevalence) / (
    sensitivity * prevalence + false_positive_rate * (1 - prevalence)
)

print(f"Sensitivity: {sensitivity:.4f}")
print(f"Specificity: {specificity:.4f}")
print(f"Prevalence : {prevalence:.6f}")
print(f"PPV      : {ppv:.4%}")
```

```
Sensitivity: 0.9990
Specificity: 0.9999
Prevalence : 0.000100
PPV      : 49.9775%
```

1.10 Problem # 2.27.

The distribution of $X = \text{heights (cm)}$ of women in the U.K. is approximately $N(162, 7^2)$. Conditional on $X = x$, suppose $Y = \text{weight (kg)}$ has a $N(3.0 + 0.40x, 8^2)$ distribution. Simulate and plot 1000 observations from this approximate bivariate normal distribution. Approximate the marginal means and standard deviations for X and Y . Approximate and interpret the correlation.

Your answer goes here

```
[19]: # Simulate 1000 observations from:
# X ~ N(162, 7^2)
# Y | X=x ~ N(3.0 + 0.40x, 8^2)
# Then plot and compute marginal means/SDs and correlation.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
n = 1000
```

```

# 1) Simulate X
X = np.random.normal(loc=162, scale=7, size=n)

# 2) Simulate Y conditional on X
mu_Y_given_X = 3.0 + 0.40 * X
Y = np.random.normal(loc=mu_Y_given_X, scale=8, size=n)

# 3) Put into dataframe
df = pd.DataFrame({"Height_cm": X, "Weight_kg": Y})

# 4) Marginal summaries
mean_X = df["Height_cm"].mean()
sd_X = df["Height_cm"].std(ddof=1)
mean_Y = df["Weight_kg"].mean()
sd_Y = df["Weight_kg"].std(ddof=1)
corr_XY = df["Height_cm"].corr(df["Weight_kg"])

print(f"Approximate marginal mean of X (height): {mean_X:.2f}")
print(f"Approximate marginal SD of X (height): {sd_X:.2f}")
print(f"Approximate marginal mean of Y (weight): {mean_Y:.2f}")
print(f"Approximate marginal SD of Y (weight): {sd_Y:.2f}")
print(f"Approximate correlation Corr(X,Y): {corr_XY:.3f}")

# 5) Plot: scatter + fitted line
plt.figure(figsize=(8,6))
sns.scatterplot(data=df, x="Height_cm", y="Weight_kg", alpha=0.55, s=35)
sns.regplot(data=df, x="Height_cm", y="Weight_kg", scatter=False, color="red")
plt.title("Simulated Bivariate Data: Height vs Weight (n=1000)")
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.grid(alpha=0.3)
plt.show()

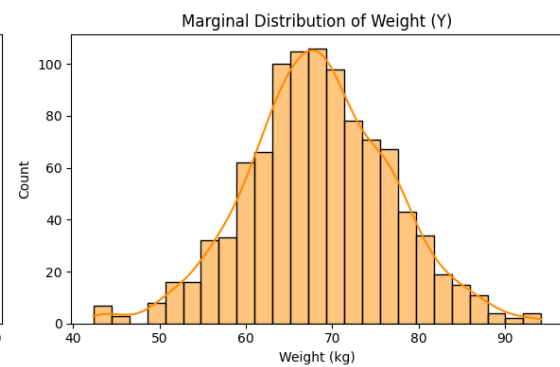
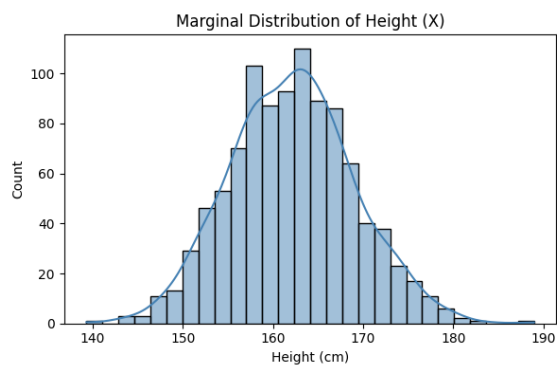
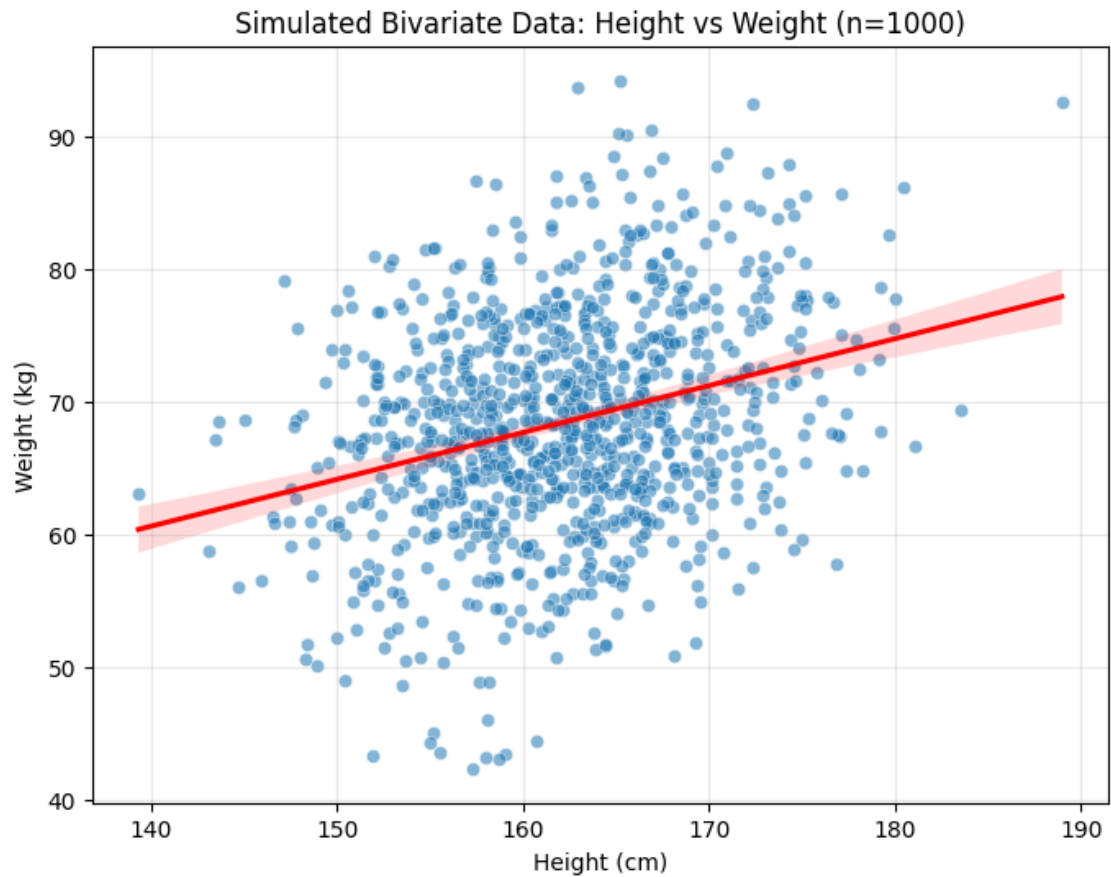
# 6) Optional: marginal histograms
fig, axes = plt.subplots(1, 2, figsize=(12,4))
sns.histplot(df["Height_cm"], kde=True, ax=axes[0], color="steelblue")
axes[0].set_title("Marginal Distribution of Height (X)")
axes[0].set_xlabel("Height (cm)")

sns.histplot(df["Weight_kg"], kde=True, ax=axes[1], color="darkorange")
axes[1].set_title("Marginal Distribution of Weight (Y)")
axes[1].set_xlabel("Weight (kg)")

plt.tight_layout()
plt.show()

```

Approximate marginal mean of X (height): 162.14
Approximate marginal SD of X (height): 6.85
Approximate marginal mean of Y (weight): 68.42
Approximate marginal SD of Y (weight): 8.33
Approximate correlation $\text{Corr}(X,Y)$: 0.290



1.11 Bayes' Theorem and AI Decision Reliability

An autonomous drone uses an onboard AI to detect whether an area contains a **target object** (for example, a missing person wearing a bright jacket).

The system raises an alert when it believes a target is present. However, the model is not perfect.

- $P(\text{Target}) = 0.03$
- $P(\text{Alert} \mid \text{Target}) = 0.95$
- $P(\text{Alert} \mid \text{No Target}) = 0.10$

1.11.1 Tasks

1. Compute the probability of an alert

Use the law of total probability:

$$P(\text{Alert}) = P(\text{Alert} \mid \text{Target})P(\text{Target}) + P(\text{Alert} \mid \text{No Target})P(\text{No Target})$$

2. Compute the posterior probability

Apply Bayes' theorem.

$$P(\text{Target} \mid \text{Alert}) = \frac{P(\text{Alert} \mid \text{Target})P(\text{Target})}{P(\text{Alert})}$$

3. Implement in Python

Write a short script that calculates and prints both probabilities with clear labels.

4. Interpret your result

In a short paragraph (as a code comment), explain what your computed ($P(\text{Target} \mid \text{Alert})$) means about the AI's reliability when it triggers an alert.

5. Bonus

Create a function that allows a user to change:

- $P(\text{Target})$
- $P(\text{Alert} \mid \text{Target})$
- $P(\text{Alert} \mid \text{No Target})$

Then return the updated posterior probability and show how the result changes when $P(\text{Target}) = 0.005$.

1.11.2 Starter Code

```
[20]: ## Bayes' Theorem in AI Detection Reliability

##### Given probabilities #####
P_target = 0.03
P_alert_given_target = 0.95
P_alert_given_no_target = 0.10
#####
```



```

# Compute complements
P_no_target = 1 - P_target

# 1) Total probability of alert
P_alert = (P_alert_given_target * P_target) + (P_alert_given_no_target *
    ↪P_no_target)

# 2) Posterior probability using Bayes' theorem
P_target_given_alert = (P_alert_given_target * P_target) / P_alert

print(f"P(Alert): {P_alert:.4f}")
print(f"P(Target | Alert): {P_target_given_alert:.4f}
    ↪({P_target_given_alert*100:.2f}%)")

# Interpretation:
# P(Target | Alert) is the probability a real target is present when an alert
    ↪is triggered.
# Here, even though sensitivity is high (0.95), the base rate of true targets
    ↪is low (0.03),
# and false alerts occur with probability 0.10 when no target is present.
# So a triggered alert is informative but not definitive; operationally, alerts
    ↪should be
# followed by secondary verification (e.g., another sensor pass or human
    ↪confirmation).

# Bonus: parameterized function returning both P(Alert) and posterior
def bayes_detection(P_target, P_alert_given_target, P_alert_given_no_target):
    P_no_target = 1 - P_target
    P_alert = (P_alert_given_target * P_target) + (P_alert_given_no_target *
    ↪P_no_target)
    P_target_given_alert = (P_alert_given_target * P_target) / P_alert
    return P_alert, P_target_given_alert

# Bonus scenario: rarer target prevalence
P_alert_new, posterior_new = bayes_detection(0.005, 0.95, 0.10)

print(f"\nWith P(Target)=0.005:")
print(f"P(Alert): {P_alert_new:.4f}")
print(f"P(Target | Alert): {posterior_new:.4f} ({posterior_new*100:.2f}%)")

```

P(Alert): 0.1255
 P(Target | Alert): 0.2271 (22.71%)

With P(Target)=0.005:
 P(Alert): 0.1043
 P(Target | Alert): 0.0456 (4.56%)